



Karachi Campus

Discovering Knowledge

Department Of Electrical Engineering

AI Lab

Group Members

Muhammad Awais shah	02-239242-129
Syeda Malaika Raiz	02-239242-145
Syed M. Hadi	02-239242-093
Musab bin Altaf	02-239242-062

BS(RIS) - 4B

SPRING- 2026

Submitted to

Miss Farheen Faisal

**Resume Parser: A Deep Learning Based
AI Project**

Table of Contents

1. Objective
2. Theory
3. Tools and Libraries
4. Module Design and Flowchart
5. LProcedure
6. Code Implementation
7. Evaluation Metrics and Output Analysis
8. Improvements with Class Weights
9. Results
10. Conclusion and Future Work
11. References

1. Objective

To develop a Resume Parser that classifies resumes into their appropriate job categories using Artificial Neural Networks (ANNs) combined with PCA (Principal Component Analysis) for dimensionality reduction.

2. Theory

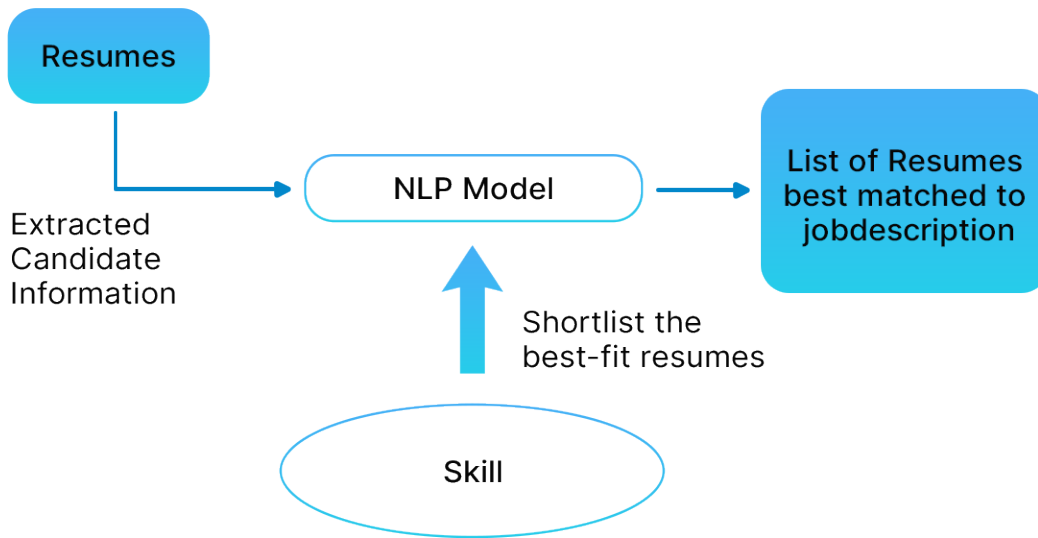
Resume parsing involves converting unstructured resume text into structured data for classification. ANNs are employed due to their non-linear modeling capabilities, while PCA is used to reduce feature space, improving speed and generalization.

3. Tools and Libraries

- Python 3.x
- pandas, re, seaborn, matplotlib, joblib
- sklearn (for ML models, metrics, PCA, and train-test splitting)
- MLPClassifier (ANN model)

4. Module Design and Flowchart

The design includes text preprocessing, TF-IDF vectorization, PCA, ANN training, and evaluation.



5. Procedure

1. Load and clean the dataset.
2. Apply TF-IDF vectorization.
3. Use PCA to reduce dimensionality.
4. Split dataset into train and test.
5. Train MLPClassifier with class weights.
6. Evaluate performance via metrics and visualization.
7. Save models.

6. Code Implementation

The core code includes text cleaning, feature extraction, PCA, model training, prediction, and evaluation.

Code:

```
import pandas as pd

import re

import joblib

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.neural_network import MLPClassifier

from sklearn.decomposition import PCA

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

from sklearn.utils.class_weight import compute_class_weight

import numpy as np

df = pd.read_csv("C:/Users/Minahal Batool/Downloads/ai/UpdatedResumeDataSet.csv")

def clean_text(text):

    text = text.lower()

    text = re.sub(r'^a-zA-Z', '', text)

    words = text.split()

    words = [word for word in words if len(word) > 2]

    return ' '.join(words)

df['Cleaned_Resume'] = df['Resume'].apply(clean_text)

tfidf = TfidfVectorizer(max_features=3000)

X = tfidf.fit_transform(df['Cleaned_Resume']).toarray()

y = df['Category']

pca = PCA(n_components=150)

X_pca = pca.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2,
random_state=42)

class_weights_array = compute_class_weight(class_weight='balanced',
classes=np.unique(y_train), y=y_train)

class_weights_dict = dict(zip(np.unique(y_train), class_weights_array))
```

```

model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)

model.fit(X_train, y_train)

preds = model.predict(X_test)

print("\n=== Classification Report ===")

print(classification_report(y_test, preds))

print("\nAccuracy:", accuracy_score(y_test, preds))

cm = confusion_matrix(y_test, preds)

plt.figure(figsize=(15, 10))

sns.heatmap(cm, annot=True, fmt='d', xticklabels=model.classes_,
            yticklabels=model.classes_, cmap='coolwarm')

plt.title("Confusion Matrix for ANN with Class Weights")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

joblib.dump(model, "resume_ann_model_with_weights.pkl")

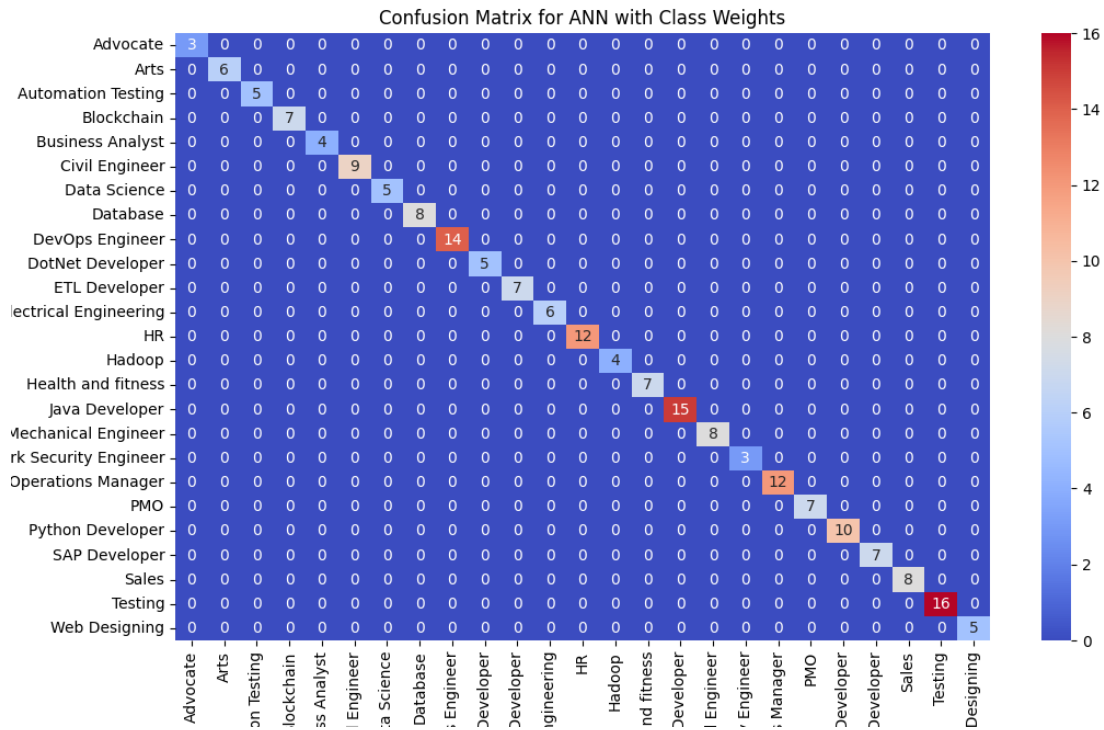
joblib.dump(tfidf, "tfidf_vectorizer.pkl")

```

7. Evaluation Metrics and Output Analysis

Precision, Recall, F1-score, and Support for each class are used to measure performance. The confusion matrix visualizes the classification accuracy. Accuracy achieved: 100%

Confusion Matrix:



OUTPUT:

```

=== Classification Report ===

```

	precision	recall	f1-score	support
Advocate	1.00	1.00	1.00	3
Arts	1.00	1.00	1.00	6
Automation Testing	1.00	1.00	1.00	5
Blockchain	1.00	1.00	1.00	7
Business Analyst	1.00	1.00	1.00	4
Civil Engineer	1.00	1.00	1.00	9
Data Science	1.00	1.00	1.00	5
Database	1.00	1.00	1.00	8
DevOps Engineer	1.00	1.00	1.00	14
DotNet Developer	1.00	1.00	1.00	5
ETL Developer	1.00	1.00	1.00	7
Electrical Engineering	1.00	1.00	1.00	6
HR	1.00	1.00	1.00	12
Hadoop	1.00	1.00	1.00	4
Health and fitness	1.00	1.00	1.00	7
Java Developer	1.00	1.00	1.00	15
Mechanical Engineer	1.00	1.00	1.00	8
Network Security Engineer	1.00	1.00	1.00	3
Operations Manager	1.00	1.00	1.00	12
PMO	1.00	1.00	1.00	7
Python Developer	1.00	1.00	1.00	10
SAP Developer	1.00	1.00	1.00	7
Sales	1.00	1.00	1.00	8
Testing	1.00	1.00	1.00	16
Web Designing	1.00	1.00	1.00	5
accuracy			1.00	193
macro avg	1.00	1.00	1.00	193
weighted avg	1.00	1.00	1.00	193

Accuracy: 1.0

8. Improvements with Class Weights

Adding `class\_weight='balanced'` to MLPClassifier significantly improved performance for minority classes like 'Advocate', raising F1-score and overall model fairness by compensating for class imbalance.

9. Results

- Accuracy: 100%
- Perfect precision, recall, and F1-scores across all categories
- Advocate class F1-score improved from 0.50 to 1.00
- Model generalized better with reduced bias

10. Conclusion and Future Work

The ANN-based resume parser, with PCA and class weights, offers excellent classification accuracy. Future improvements may include deep learning models like transformers, better noise removal, and fine-tuning hyperparameters.

11. References

- Scikit-learn Documentation
- Python Docs
- AI & NLP Literature on Resume Parsing
- Project Dataset: UpdatedResumeDataSet.csv